SHARE: Smart Hostel Allocation and Room Exchange

A Data Structures Approach to Fair and Transparent Hostel Room Allotment

Team Name: Coffee Overflow

Rohit 106125097
Sagar Aggarwal 106125099
Sanapala Rahul 106125101

Department of Computer Science and Engineering
National Institute of Technology, Tiruchirappalli

CSLR31 – Data Structures Laboratory

Aligned with UN Sustainable Development Goals 4 (Quality Education) and 11 (Sustainable Cities and Communities)

Project Report
November 2026

Contents

1 Abstract **3**

2 Design of the Product **4**
2.1 The idea . . . 4
2.2 Subsystem A: Hostel Room Allocator . . . 4
2.3 Subsystem B: Room Exchange Module . . . 6
2.4 How it is implemented . . . 6

3 Coding **8**
3.1 Hostel Room Allocator . . . 8
3.1.1 Stage 0: Setting up the room inventory . . . 8
3.1.2 Stage 1: Student registration . . . 8
3.1.3 Stage 2: Forming roommate pairs . . . 8
3.1.4 Stage 3: The lottery . . . 9
3.1.5 Stage 4: Allocating rooms . . . 9
3.2 Room Exchange Module . . . 11
3.2.1 Data and state used by the exchanger . . . 11
3.2.2 Creating a room exchange . . . 11
3.2.3 Creating a roommate swap . . . 11
3.2.4 Student login and responding to requests . . . 12
3.2.5 Execution of an exchange . . . 12

4 Testing **13**
4.1 Test data . . . 13
4.2 Hostel Room Allocator . . . 13
4.3 Room Exchange Module . . . 16

5 Contributions **18**

6 Key Takeaways **18**

7 Future Scope **19**

8 Acknowledgements **19**

9 References **19**

1 Abstract

Hostel room allotment in most Indian institutes, including NIT Tiruchirappalli, is still carried out through spreadsheets, printed notice boards and informal messaging. The process is slow at the start of every semester and opaque throughout. A student has no way of knowing why a particular room was assigned, and requests to change rooms or roommates afterwards are settled as informal favours between whoever happens to find each other. Existing products do not close this gap. Education ERPs such as Fedena and eduSec do ship hostel modules, but they treat allotment as a bookkeeping entry, a form recording which student occupies which room, rather than as a problem to be computed, and room exchange, where it exists at all, is a manual edit by an administrator. SHARE is a modular command-line system, written in C, that treats hostel allotment as a data structures problem. It has two subsystems that run one after the other. The Hostel Room Allocator takes each student's three ranked (hostel, floor) choices and an optional roommate, pairs the students who asked for a random roommate by how well their choices overlap, runs a lottery, and places every pair in the best floor that still has a free room. The Room Exchange Module then lets students change rooms with the agreement of everyone affected, through a mutual exchange between two rooms, a roommate swap with a named student, or a random pool in which students list floors they would rather be on. Pool requests form a directed graph whose nodes are floors and whose edges are students wishing to move. A cycle in this graph is a ring of students who can each take the next one's room with no floor gaining or losing anybody, so cycle detection finds not only two-way swaps but the three-way and longer rotations that a manual process could never find. Every request is applied completely or not at all, and every completed exchange is appended to a persistent audit log. Students are kept in a hash table keyed on roll number with linked-list chaining, and all state is saved to file so it survives across runs. The implementation follows the modular convention required by the course, with separate `.h` declarations and `.c` definitions per module and a Makefile-driven build. Future scope includes moving into vacant rooms, a maximum-matching method for the pool, a graphical front end in place of the command-line interface, and integration with the institute ERP for authentication.

2 Design of the Product

2.1 The idea

SHARE replaces two things the hostel office does by hand today. The first is the yearly allotment of rooms, where students hand in floor preferences and a roommate's name and somebody works through the list. The second is the stream of requests afterwards from students who want to swap rooms or roommates, which today goes through the warden one case at a time. The program handles both. It takes registrations and produces an allocation table, and then lets students arrange exchanges among themselves with the agreement of everyone affected, so that every change is recorded and nothing is applied halfway.

The program is a console application in C with no background process. It is split into two subsystems that run one after the other and talk to each other through one file, the allocation table. Each subsystem is three packages, one per member, with a header file for the interface so the others could build against it. Figure 1 shows the whole flow, with the allocator on the left and the exchange module on the right.

2.2 Subsystem A: Hostel Room Allocator

The allocator runs once, from the top of the left column of the flowchart to the bottom.

Loading. The vacant-room list and the eligible-student list are read in. The room list gives the inventory, how many rooms are free on each floor of each hostel. The student list is the set of roll numbers allowed to register, and anyone not on it is refused at entry.

Registration window. A student enters three ranked (hostel, floor) choices and either a roommate's roll number or RANDOM. Each entry is validated as it is typed, and an invalid entry goes back to the form. The roommate status is then set. Naming someone makes the student PENDING, and when the other student names them back, both become LOCKED. Asking for RANDOM goes straight to RANDOM. Students can edit or unlock their entry as long as the window is open. Registrations are kept in a hash table keyed by roll number, with a singly linked list for the chains, and are written to file when the window is closed and when the program exits.

Closing the window. When the admin closes the window, every student still PENDING becomes RANDOM, since their named roommate never confirmed, and the list of eligible students who never registered is written out so they can be placed at the end.

Pairing the random pool. The random students are paired by a compatibility score that rewards shared floors over shared hostels and higher-ranked matches over lower ones. The scores are sorted with a counting sort and pairs are taken greedily from the best score down. Each new pair is given a shared list of three floors built from the two students' lists. Locked pairs are checked (both records exist, both LOCKED, each names the other) and become units directly. The result is a list of units, each a pair with up to three floor choices. The number of students is assumed to be even, so there are no solo units. A demand report is then printed showing free rooms against first-choice demand per floor.

Lottery. The paired file has two lines per pair, one for each student, so the lottery stage first merges them into one record per pair, sorted by roll number. The pairs are then shuffled with a Fisher–Yates shuffle, seeded from the clock, and written out in lottery order.

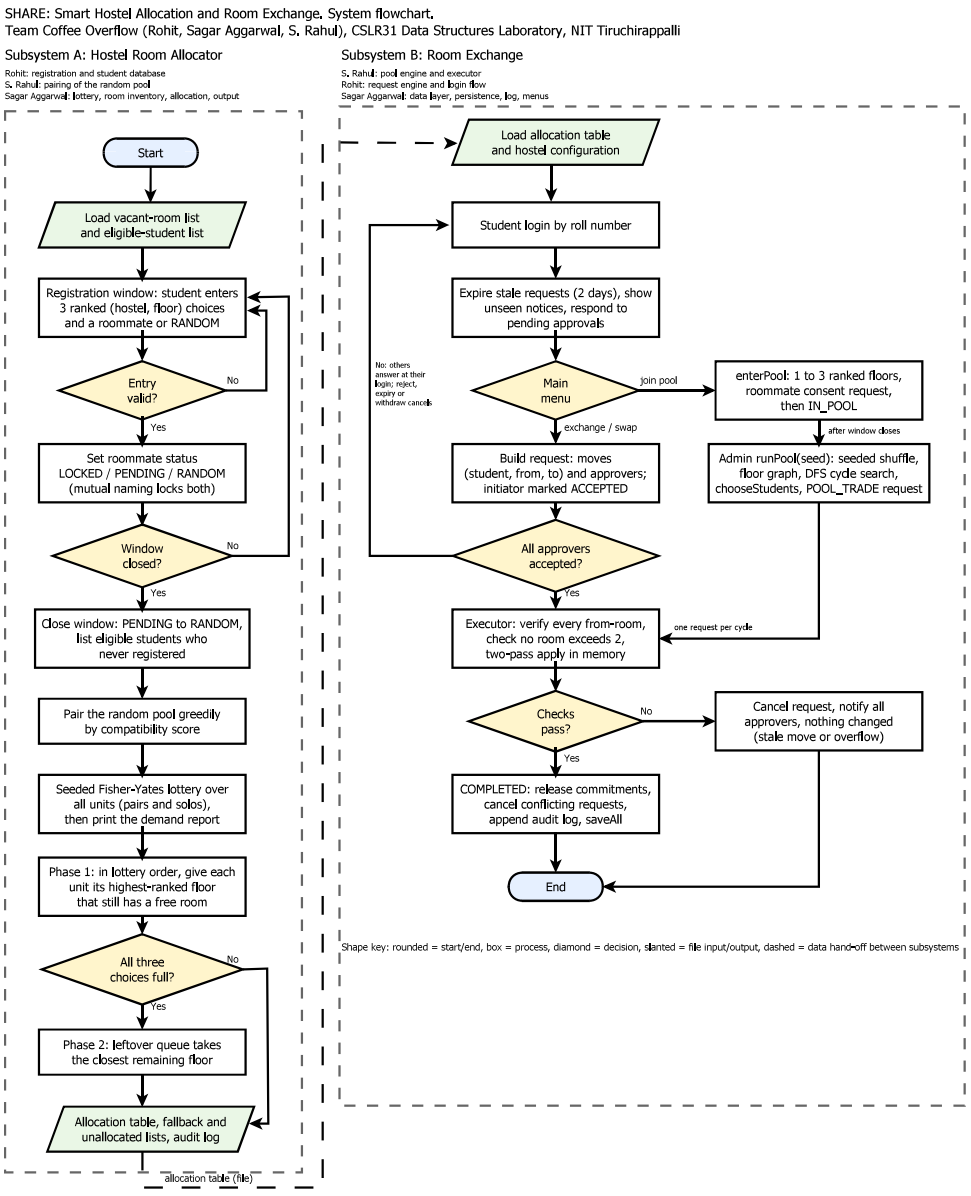


Figure 1: System flowchart. Subsystem A on the left is the Hostel Room Allocator, Subsystem B on the right is the Room Exchange Module. The dashed arrow is the allocation table handed from one to the other.

Allocation. In lottery order, each pair is given its highest-ranked floor that still has a free room (phase 1). Pairs whose three choices are all full go into a leftover list, and when the first pass is done that list is served in order (phase 2). For each of the pair's choices the nearest floor above or below it in the same hostel is tried first, and if none has a room, any free floor in a hostel the pair did not list. The output is the allocation table, one line per pair with hostel, floor and room, and a count of how many pairs went through phase 2.

2.3 Subsystem B: Room Exchange Module

The exchange module starts from the allocation table. Everything a student needs to see or answer appears when they log in.

Login. The student enters their roll number. Any of their requests older than two days are expired, their unseen notices are shown once, and each open request waiting for their answer is shown with the choice Accept, Reject or Skip for later. Then the main menu appears.

Requests. A mutual room exchange swaps two rooms, with both pairs keeping their roommate. A roommate swap trades two named students so each ends up with the other's roommate. Both are built the same way, a list of moves of the form (student, from room, to room) and a list of approvers, with the initiator already marked ACCEPTED. The others see it at their next login. A student can hold one open request at a time, which is the commitment rule, and cannot be named in another request while it is open. One rejection, a withdrawal by the initiator, or expiry cancels the request and releases everyone.

The random pool. A student who would rather be on a different floor lists one to three floors, best first. If they share a room, their roommate has to consent first, through a request like any other. The admin runs the pool once with a seed, after students have had time to join. The pool students are shuffled, then a graph is built whose nodes are floors and whose arrows go from each student's floor to the floors they listed. A cycle in that graph is a ring of students who can each move into the next one's room without any floor gaining or losing anybody. Rings are found with a depth-first search, first choices before second choices, and each ring becomes one POOL_TRADE request.

The executor. Every request, from any of the three features, ends up in the executor once all approvers have accepted. It checks that each student is still in the room the move says and that no room would end up with more than two people. If every check passes, the moves are applied in two passes, everyone out first and then everyone in, so a ring of three or more rooms never has a room at three people part way through. The request is then COMPLETED, commitments are released, any other open request on the same rooms is cancelled with a notice, one line goes to the audit log and everything is saved. If a check fails, the request is cancelled with the reason and nothing has changed.

2.4 How it is implemented

Both subsystems are built the same way. The shared records (student, room, unit, request, move, approver, notice) are defined in a common header that nobody changes without telling the others. Each member's package is a header with its function declarations and a source file with the definitions, and during development each package carried small stand-ins for the other two so it could be built and tested on its own. Integration was a matter of replacing the stand-ins with the real files and wiring the menus.

The data structures are chosen by how the data is used. Students are looked up by roll number, so in both subsystems they sit in a hash table with chained singly linked lists. Rooms and requests have small dense ids, so they sit in arrays indexed by id. Open requests on a room are a list that grows and shrinks, so it is a linked list. The compatibility scores take only 17 values, so a counting sort orders them in linear time. The leftover pairs in allocation are served in the order they were set aside. The floor graph in the pool is small, nine floors, so it is an adjacency table with a colour array, and the depth-first search keeps its path on an explicit stack. Both the lottery and the pool use a Fisher–Yates shuffle. The pool's is driven by an admin-supplied seed and our own generator, so a run can be repeated exactly.

Every package has a test file that builds small scenarios in memory and checks the results, run with `make test`. The tests are described in Section 4.

3 Coding

3.1 Hostel Room Allocator

The allocator takes students from registration to their final room assignments. It does this in a few steps: first it records students and their preferences, then it forms roommate pairs, decides the order in which those pairs will be considered, and then finally assigns rooms.

3.1.1 Stage 0: Setting up the room inventory

The program imports the vacant rooms from the vacant.txt txt file and stores the available rooms in linked lists. Each hostel-floor combination has its own list. The first node keeps track of how many rooms are available, while the remaining nodes contain the actual room numbers. When a room is allocated, the program removes its node from the list and reduces the vacancy count.

3.1.2 Stage 1: Student registration

This stage is where students provide their roll number, roommate choice, and their preferred hostel-floor combinations.

The program uses a hash table with 100 buckets to look up students using their roll numbers. The hash function is

$$h(\text{roll number}) = \text{roll number} \bmod 100$$

collisions are dealt by separate chaining method. This makes student lookup quick on average, instead of having to go through the entire student list every time.

Handling roommate choices. A student can choose either a random roommate or enter the roll number of someone they want to share a room with. The program keeps track of roommate requests using three statuses:

- RANDOM: the student is open to being paired with someone randomly.
- PENDING: the student has requested a particular roommate, but the match is not yet mutual.
- LOCKED: both students have selected each other, so their roommate pair is confirmed.

When two students mutually select each other, they become a LOCKED pair. Their hostel-floor preferences are also made the same.

Checking hostel preferences. Each student enters three hostel-floor choices in order of priority. The program ensures that the details are valid and the choice entries are not duplicate.

When registration closes, roomate requests that are still PENDING are changed to RANDOM.

3.1.3 Stage 2: Forming roommate pairs

This stage takes the registered students and turns them into allocation units. An allocation unit is usually a pair of students who will share a room.

There are two main groups to handle: students who have already confirmed their roommates, and students who still need to be paired.

Keeping confirmed pairs together. The program first checks the LOCKED pairs. If the pair is valid, it is added as one allocation unit. The program creates the pair only once, using the student with the lower roll number.

Giving RANDOM students a compatibility score. For students who still need roommates, the program compares every possible pair. It checks the three preferences of one student against the three preferences of the other. This gives nine possible comparisons. A pair gets a higher score if both students selected the same hostel and floor; a smaller score if they selected different floors in the same hostel; zero if they do not share a hostel in their preference lists.

When the students share a floor, the score also depends on how highly each student ranked that floor. A floor that both students ranked near the top receives a better score.

Greedy algorithm for pairing students. Once all possible pairs have scores, the program tries to form the most compatible pairs first. This is a greedy approach because the program chooses the best available pair at each step rather than checking every possible complete pairing. It is straightforward and practical, although it does not guarantee the mathematically best overall combination of pairs.

Creating a shared preference list. After two students are paired, they need one common list of hostel-floor preferences. The program combines the preferences from both students and removes duplicates. It then gives each candidate floor a combined rank: Combined rank = Student A's rank + Student B's rank. If a floor does not appear in one student's list, it is given rank 3 for that student. The program uses Insertion Sort to put the candidates in order, with the lowest combined rank first. If two candidates have the same rank, hostel code and floor number are used to decide their order. The best three candidates become the pair's shared preference list.

3.1.4 Stage 3: The lottery

Once the allocation units have been formed, the program needs to decide the order in which they get a chance to choose rooms. It uses the Fisher-Yates Shuffle to randomize the allocation units. The idea is simple: - Start at the last element of the array; pick a random position from the beginning of the array up to the current position; swap the two elements; move one position backwards and repeat.

Fisher-Yates takes $O(n)$ time for n allocation units. When the random number generation is uniform, it produces a uniformly random ordering. The use of a seed also makes it possible to reproduce a particular lottery order when the same seed and input are used.

The purpose of this stage is to give allocation units a randomized order, rather than allowing registration order to decide who gets considered first.

3.1.5 Stage 4: Allocating rooms

This is the final stage. The program takes the allocation units in lottery order and tries to find a room for each one.

The allocation happens in two phases:

Phase 1: Trying the students' preferences. The program checks each allocation unit's preferences in the order they were submitted.

For each unit:

1. It checks the first preferred hostel and floor.
2. If a room is available, it assigns one.
3. If not, it checks the second preference.

4. If that is also full, it checks the third preference.
5. If none of the three preferences has a vacant room, the unit is kept in an unallocated list for the next phase.

So basically ach unit gets the highest-ranked preference that is still available when the program reaches it or else it is added to the phase 2 pool

Phase 2: Finding an alternative room. Some units may not get a room in Phase 1 because all three of their preferred floors are full. Phase 2 tries to find them another option.

First, the program searches nearby floors in the hostels from the unit's preference list. For each preferred floor, it checks one floor above and below; two floors above and below; and so on. If no room is available on these nearby floors, the program moves on to the other hostels. It skips hostels that were already included in the unit's preference list and checks the remaining hostels for vacant rooms. The first available room it finds is assigned.

3.2 Room Exchange Module

The exchanger allows students who already have hostel rooms to request changes in room occupancy. It supports two kinds of exchanges:

1. Room exchange The occupants of two rooms exchange rooms as complete groups.
2. Roommate swap Two selected students exchange places, while their current roommates remain in their original rooms.

An exchange is not carried out immediately after a request is created. The people whose approval is required must respond first. The system executes the exchange only when all required approvals have been received and the request still passes its validation checks.

3.2.1 Data and state used by the exchanger

The exchanger works with student records, room records, exchange requests, and approver records.

A request stores information including the request identifier, the type of exchange, the students who are supposed to move, the rooms involved, the students whose approval is required, the approval status of each approver and the overall request status.

Request status values:-

1. REQ_OPEN = 0: The request is still waiting for the required responses or execution.
2. REQ_COMPLETED = 1: The exchange was successfully carried out.
3. REQ_CANCELLED = 2: The request was cancelled.
4. REQ_EXPIRED = 3: The request expired.

Approver status values:-

1. APPROVER_PENDING = 0: The student has not responded yet.
2. APPROVER_ACCEPTED = 1: The student has accepted.
3. APPROVER_REJECTED = 2: The student has rejected.

The system uses these states to decide whether a request can proceed.

3.2.2 Creating a room exchange

Function: `createExchangeRoom()`

A room exchange moves all occupants of one room into another room, and all occupants of the target room into the first room.

All occupants of both rooms are included as approvers.

3.2.3 Creating a roommate swap

Function: `createRoomateSwap()`

A roommate swap exchanges the rooms of two selected students. Their existing roommates do not move.

All occupants of both rooms are approvers, including the two students who are swapping.

3.2.4 Student login and responding to requests

The login/request interaction allows a student to view requests that are waiting for their approval.

The student can choose one of the available responses:

1. ACCEPT: Approve the request.
2. REJECT: Reject the request.
3. SKIP FOR LATER: Leave the request pending so that it can be considered later.

What happens after a response?

- If the student accepts, their approver state becomes APPROVER_ACCEPTED.
- If the student rejects, the request is cancelled.
- If the student skips, the request remains open and that approver remains pending.

A request cannot be executed while a required approver is still pending.

3.2.5 Execution of an exchange

Function: `execute()`

The executor performs the actual room changes after the request has received all required approvals.

The main checks and operations are:

1. Check approvals. The executor checks whether every required approver has accepted.
2. Check that the request is still valid. The executor checks that the students and rooms involved still match the expected move.
3. Update the request. The request is marked as REQ_COMPLETED. The system sends the relevant notifications. Conflicting requests may be cancelled. The system writes the relevant logs and saves the updated data.

4 Testing

All tests use one data set, the 30 registrations in `student_hostel_choices.txt` (out of 720 eligible students in `EligibleStudentfile.txt`), and the room inventory in `vacant_rooms.txt` with 360 rooms (AA floors 0 to 3 and AB floors 0 to 3 with 40 rooms each, GC floor 2 with 40 rooms). The allocator was run stage by stage on this input and its allocation was then the starting point for the exchange module tests, which follow `testCaseOverview.txt`.

4.1 Test data

Table 1 is the registration list as recorded by Stage 1. Twenty students locked with a named roommate (ten pairs) and ten asked for a random roommate. The ten random students were chosen so that their choices overlap in different ways, which is what the pairing has to sort out.

Roll	Name	Choice 1	Choice 2	Choice 3	Status	Roommate
106125001	Aarav Sharma	AA0	AA1	AA2	LOCKED	106125002
106125002	Aarav Patel	AA0	AA1	AA2	LOCKED	106125001
106125003	Aarav Reddy	AA2	AA3	AB0	LOCKED	106125004
106125004	Aarav Iyer	AA2	AA3	AB0	LOCKED	106125003
106125005	Aarav Nair	AB0	AB1	AB2	LOCKED	106125006
106125006	Aarav Rao	AB0	AB1	AB2	LOCKED	106125005
106125007	Aarav Menon	AB2	AB3	GC2	LOCKED	106125008
106125008	Aarav Kumar	AB2	AB3	GC2	LOCKED	106125007
106125009	Aarav Singh	GC2	AA0	AA1	LOCKED	106125010
106125010	Aarav Pillai	GC2	AA0	AA1	LOCKED	106125009
106125011	Aarav Verma	AA1	AA2	AA3	LOCKED	106125012
106125012	Aarav Krishnan	AA1	AA2	AA3	LOCKED	106125011
106125013	Aditya Sharma	AA3	AB0	AB1	LOCKED	106125014
106125014	Aditya Patel	AA3	AB0	AB1	LOCKED	106125013
106125015	Aditya Reddy	AB1	AB2	AB3	LOCKED	106125016
106125016	Aditya Iyer	AB1	AB2	AB3	LOCKED	106125015
106125017	Aditya Nair	AB3	GC2	AA0	LOCKED	106125018
106125018	Aditya Rao	AB3	GC2	AA0	LOCKED	106125017
106125019	Aditya Menon	AA0	AA1	AA2	LOCKED	106125020
106125020	Aditya Kumar	AA0	AA1	AA2	LOCKED	106125019
106125021	Aditya Singh	AA2	AA3	AB0	RANDOM	
106125022	Aditya Pillai	AA3	AB0	AB1	RANDOM	
106125023	Aditya Verma	AB0	AB1	AB2	RANDOM	
106125024	Aditya Krishnan	AB1	AB2	AB3	RANDOM	
106125025	Aishwarya Sharma	AB2	AB3	GC2	RANDOM	
106125026	Aishwarya Patel	AB3	GC2	AA0	RANDOM	
106125027	Aishwarya Reddy	GC2	AA0	AA1	RANDOM	
106125028	Aishwarya Iyer	AA0	AA1	AA2	RANDOM	
106125029	Aishwarya Nair	AA1	AA2	AA3	RANDOM	
106125030	Aishwarya Rao	AA2	AA3	AB0	RANDOM	

Table 1: Registrations used in every test. Floors are numbered from 0.

4.2 Hostel Room Allocator

T1. Pairing of the random pool (Stage 2). The input is the 30 registrations and the list of 690 students who never registered. We expected the ten locked pairs to pass the mutual-naming check and become units unchanged, the ten random students to be paired by compatibility score with the best scores first, each new pair to get a shared choice list, and the 690 unregistered students to be appended as 345 units with no preferences. That is what happened. The program printed `units 15 (15 pairs, 0 solo)` for the registered students and `units 360 (345`

`without preferences), rooms 360` after the unregistered ones were added. The random pairs it made are in Table 2, with the score the greedy pass used. The pair 021 and 030 has identical lists, which is the maximum score of 16, and was taken first. Every remaining pair scored 15 (a shared first-or-second choice), and the tie-break is the generation order after sorting by roll number.

Student A	Student B	Score	Shared choices
106125021	106125030	16	AA2 AA3 AB0
106125022	106125023	15	AB0 AA3 AB1
106125024	106125025	15	AB2 AB1 AB3
106125026	106125027	15	GC2 AA0 AB3
106125028	106125029	15	AA1 AA0 AA2

Table 2: Random students as paired by Stage 2.

T2. Demand report (Stage 2). The report should show, for each floor, the free rooms and the number of units that listed it first, with a warning where demand is above supply. It did. With 40 rooms per floor and at most two first choices per floor nothing is oversubscribed, so no warning was printed.

Hostel	Floor	Rooms	First choice
AA	0	40	2
AA	1	40	2
AA	2	40	2
AA	3	40	1
AB	0	40	2
AB	1	40	1
AB	2	40	2
AB	3	40	1
GC	2	40	2

T3. Lottery (Stage 3). The input is the paired file from Stage 2, which has two lines per pair. Each pair should be merged into one record and the 15 pairs shuffled with a Fisher–Yates shuffle. Table 3 is the order this run produced. Since the shuffle is seeded from the clock, a second run gives a different order, so what is checked is that every pair appears exactly once with its shared choices intact. It did.

T4. Allocation with the full inventory (Stage 4). The input is the lottery order above and the 360-room inventory. With 40 rooms on every floor and at most two pairs wanting any floor first, every pair should get its first choice in phase 1 and nobody should reach phase 2. That is what happened. The program printed `Allocated pairs 15, pairs sent to phase 2 0`. The rooms are in Table 4. The lowest free room on the floor is taken each time, so the two pairs wanting AA floor 0 got rooms 1 and 2 in lottery order.

T5. Allocation with a tight inventory, exercising phase 2. To make phase 2 happen the inventory was cut to 15 rooms, two each on AA floor 0, AB floor 0, AB floor 1, AB floor 2 and GC floor 2, three on AA floor 1, one each on AA floor 3 and AB floor 3, and none on AA floor 2. The same lottery order was used. The two pairs that want AA floor 2 first (003/004 and 021/030) cannot get it. The earlier one in lottery order should take its second choice AA floor 3, and the later one should find its second and third choices full as well and go to phase 2, which

#	Student A	Student B	Choices
1	106125026	106125027	GC2 AA0 AB3
2	106125009	106125010	GC2 AA0 AA1
3	106125003	106125004	AA2 AA3 AB0
4	106125005	106125006	AB0 AB1 AB2
5	106125024	106125025	AB2 AB1 AB3
6	106125017	106125018	AB3 GC2 AA0
7	106125013	106125014	AA3 AB0 AB1
8	106125015	106125016	AB1 AB2 AB3
9	106125021	106125030	AA2 AA3 AB0
10	106125007	106125008	AB2 AB3 GC2
11	106125001	106125002	AA0 AA1 AA2
12	106125022	106125023	AB0 AA3 AB1
13	106125011	106125012	AA1 AA2 AA3
14	106125019	106125020	AA0 AA1 AA2
15	106125028	106125029	AA1 AA0 AA2

Table 3: Lottery order of the run used for the allocation tests.

Student A	Student B	Hostel, floor	Room	Choice got
106125026	106125027	GC 2	80	first
106125009	106125010	GC 2	81	first
106125003	106125004	AA 2	81	first
106125005	106125006	AB 0	1	first
106125024	106125025	AB 2	81	first
106125017	106125018	AB 3	121	first
106125013	106125014	AA 3	121	first
106125015	106125016	AB 1	41	first
106125021	106125030	AA 2	82	first
106125007	106125008	AB 2	82	first
106125001	106125002	AA 0	1	first
106125022	106125023	AB 0	2	first
106125011	106125012	AA 1	41	first
106125019	106125020	AA 0	2	first
106125028	106125029	AA 1	42	first

Table 4: Allocation with the full inventory, lottery order as in Table 3.

should give it the nearest floor to a choice in the same hostel. That is what happened. The program printed `Allocated pairs 15, pairs sent to phase 2 1`. Pair 021/030 was placed on AA floor 1, room 43, which is the nearest floor with a room to its first choice AA floor 2 (floor 3 was already taken by 003/004). Pair 013/014, whose first choice AA floor 3 was gone by its turn, took its second choice AB floor 0 in phase 1.

Student A	Student B	Hostel, floor	Room	Choice got
106125026	106125027	GC 2	80	first
106125009	106125010	GC 2	81	first
106125003	106125004	AA 3	121	second
106125005	106125006	AB 0	1	first
106125024	106125025	AB 2	81	first
106125017	106125018	AB 3	121	first
106125013	106125014	AB 0	2	second
106125015	106125016	AB 1	41	first
106125007	106125008	AB 2	82	first
106125001	106125002	AA 0	1	first
106125022	106125023	AB 1	42	third
106125011	106125012	AA 1	41	first
106125019	106125020	AA 0	2	first
106125028	106125029	AA 1	42	first
106125021	106125030	AA 1	43	phase 2

Table 5: Allocation with the 15-room inventory.

4.3 Room Exchange Module

The exchange module was started on the allocation of Table 4, imported from the allocator's output file on its first run. Cases E1 to E4 are the ones listed in `testCaseOverview.txt`. They all start from student 106125019, who is in AA room 2 with 106125020, and involve AB room 81, where 106125024 and 106125025 live. Each case was run from a fresh copy of the allocation.

#	Type	Target	Responses in order	Result
E1	room exchange	room AB-81	020 accept, 024 accept, 025 accept	applied
E2	room exchange	room AB-81	020 accept, 024 reject	cancelled
E3	roommate swap	student 106125024	020 accept, 024 accept, 025 accept	applied
E4	roommate swap	student 106125024	020 accept, 024 reject	cancelled

Table 6: Request engine cases from `testCaseOverview.txt`, all started by 106125019. All four gave the expected result.

E1. Room exchange, all accept. After the third acceptance the rooms were swapped, 106125019 and 106125020 to AB-81 and 106125024 and 106125025 to AA-2. The audit log got one line, `ROOM_EXCHANGE request 1` followed by the four moves.

E2. Room exchange, one rejection. 106125024 rejected after 106125020 had accepted. Nobody moved, the request was saved as CANCELLED, and when 106125019 logged in again `View Request` said there was no open request, so the initiator was released.

E3. Roommate swap, all accept. 106125019 and 106125024 traded places. 106125019 ended in AB-81 with 106125025 and 106125024 in AA-2 with 106125020. One audit line with the two moves.

E4. Roommate swap, one rejection. Same as E2. Cancelled, nobody moved, initiator free again.

E5. Random pool. Three students joined the pool. 106125011 (AA floor 1) listed AA2, 106125003 (AA floor 2) listed AB0, and 106125005 (AB floor 0) listed AA1 then AA3. Each roommate saw a consent request at login and accepted. The admin ran the pool with seed 7. The floor graph has the cycle AA1, AA2, AB0, so one ring was expected. The run printed

```
pool run, seed 7, 3 student(s) in the pool
ring applied (request 4):
  106125011 Student_106125011    AA-41 -> AA-81
  106125003 Student_106125003    AA-81 -> AB-1
  106125005 Student_106125005    AB-1 -> AA-41
done: 1 ring(s), 3 student(s) moved, 0 without a match
```

Every room still held two afterwards and the log got a line for the run with its seed and one for the trade. The same joins and seed gave the same ring on a second run.

E6. Restart with an open request. A request was created, the program exited and started again, and the approver logged in. The request was in `requests.txt` with all its moves and approvers but was not shown at login, so this case fails at present. The reason is that an empty reason field is written as two adjacent commas and the loader uses `strtok`, which skips empty fields, so the fields after it are read one place out. Within a single run of the program the request logic is unaffected.

E7. Package tests. The pool and executor package has its own test program, `test_pool.c`, with 123 checks against an in-memory stand-in for the data layer, covering the entry rules, the consent paths, the executor cases, rings of two to four floors and seed repeatability. The pairing package has `test_pairing.c`, which checks the sample registration file against an expected pairing file. Both run with `make test`.

5 Contributions

The work was split so that each member owned one package in each subsystem, with a header file for the interface and the other members' parts stubbed so every package built and ran on its own. The same kind of work stayed with the same person in both halves. The table shows the split and the lists after it say what each of us did.

Member	Hostel Room Allocator	Room Exchange Module
Rohit (106125097)	Registration and student database	Request engine and login flow
Sanapala Rahul (106125101)	Pairing of the random pool	Pool engine and executor
Sagar Aggarwal (106125099)	Lottery, room inventory, allocation and output	Data layer, persistence, log and menus

Rohit

1. Registration, validation, the hash table and linked list, roommate locking logic, and save and load in the hostel room allocator
2. The request engine and login flow in the room exchange module

Sanapala Rahul

1. Pairing of the random pool (compatibility scoring, greedy pairing, shared preference lists) and the demand report in the hostel room allocator
2. The pool engine (floor graph, cycle search, ring selection) and the executor in the room exchange module
3. Test programs for both of the above

Sagar Aggarwal

1. Room inventory functions, duplicate pair merger and lottery algorithm in the hostel room allocator
2. Database management (saving and loading), helper functions used for accessing and editing stored structures
3. General algorithm blueprints for both modules and roles division
4. Test runs

6 Key Takeaways

- We had studied hash tables and graphs in advance, as separate topics. Using them in one program taught us when each one is actually the right choice, and that the decision usually comes from how the data will be looked up or changed, not from the structure itself.
- Splitting the program into header files and separate source files was new to us. It took time to get the interfaces agreed, but once the headers were fixed each of us could work on our own part and test it without waiting for the others.
- We learnt that the simple way often works. We spent time thinking about optimal matching for the pool and ended up with a greedy search that is easy to follow and good enough. Knowing its limits and writing them down was part of the learning.

- Writing test cases before putting the parts together caught most of our mistakes early. The ones that got through were in places where two people had understood the same rule differently, which taught us to write the rules down more carefully.
- Working with make, a seeded random generator and LaTeX for the report gave us tools we had not used before and will keep using.

7 Future Scope

- Replace the greedy ring search in the pool with a maximum-matching method on the floor graph, so the largest possible number of students move.
- Allow moves into vacant rooms in the exchange module. At present only occupied beds are traded.
- Log in through the institute ERP instead of a bare roll number, and send notices by email or SMS instead of waiting for the next login.
- A graphical or web front end over the same data files, so students do not need the console.
- Expiry by a real clock service rather than the machine clock, with a background check instead of checking only at login and admin actions.

8 Acknowledgements

We thank our professor for the assignment and for the review session at the start of the semester, where we brought three problem statements and he advised us on which one to take up. His guidance there shaped the project.

9 References

References

[1] GeeksforGeeks, *Data Structures*, `https://www.geeksforgeeks.org/data-structures/`, accessed September 2026.

[2] GeeksforGeeks, *Hashing Data Structure*, `https://www.geeksforgeeks.org/hashing-data-structure/`, accessed September 2026.

[3] GeeksforGeeks, *Detect Cycle in a Directed Graph*, `https://www.geeksforgeeks.org/detect-cycle-in-a-graph/`, accessed October 2026.

[4] GeeksforGeeks, *Shuffle a given array using Fisher–Yates shuffle algorithm*, `https://www.geeksforgeeks.org/shuffle-a-given-array-using-fisher-yates-shuffle-algorithm/`, accessed September 2026.

[5] GeeksforGeeks, *Counting Sort*, `https://www.geeksforgeeks.org/counting-sort/`, accessed September 2026.

[6] Overleaf, *Documentation*, `https://www.overleaf.com/learn`, accessed October 2026.

[7] UNESCO, *Sustainable Development Goals*, `https://www.unesco.org/en/sdgs`, accessed August 2026.

[8] Google Search, `https://www.google.com`, used throughout for looking up C library functions and error messages.